Mastering Claude Code: The Complete Architecture-Informed Guide

> Everything a software engineer needs to know about agentic AI-assisted coding — from model internals to production workflow — explained so you can understand, remember, and apply it.

Part I: Understanding the Machine

1. What You're Actually Working With

Claude Code is not autocomplete. It's not a chatbot with a code theme. It's an **autonomous agent** — a program that calls an AI model in a loop, executes real actions on your computer between each call, and stops only when it decides it's done or you intervene.

Internally codenamed "tengu", the architecture is a single-threaded agentic loop. Here's what happens every time you send a message:

1. **Assembly.** Claude Code builds a prompt from your system prompt, CLAUDE.md contents, tool definitions (including MCP servers), conversation history, and your new message.
2. **API call.** This assembled prompt is sent to Anthropic's API — passing through authentication, input moderation, inference, output moderation, and streaming. Claude Code runs locally, but it doesn't bypass any safety layers.
3. **Response parsing.** The model returns either plain text (conversation over, control returns to you) or `tool_use` blocks (the agent wants to do something).
4. **Tool execution.** If tool calls are present, Claude Code executes them locally — reading files, running bash commands, editing code — and feeds the results back to the model.
5. **Loop.** Steps 2–4 repeat until the model returns plain text with no tool calls.

Why this matters: Your prompt doesn't generate code. It *launches an autonomous agent* that will read, think, act, verify, and iterate. The quality of that entire multi-step loop depends on three things: how well you set up its context, how clearly you define the task, and how effectively you constrain what the agent can do.

The tools Claude has access to

Think of these as the agent's hands and eyes:

- **Read tools** — Read, Glob, Grep, LS, NotebookRead. These are how Claude *sees* your codebase. Glob finds files by pattern, Grep searches within files, Read loads file contents, LS lists directories.
- **Write tools** — Edit, MultiEdit, Write, NotebookEdit. These are how Claude *changes* your code. Edit modifies specific sections, Write creates or replaces files.
- **Execution** — Bash. This is Claude's most powerful tool. It runs commands in persistent shell sessions — tests, builds, git operations, Docker, package managers, anything your terminal can do.
- **Network** — WebFetch, WebSearch. Claude can read web pages and search the internet for documentation, API references, or solutions.
- **Planning** — TodoRead, TodoWrite. Structured task tracking that persists as a system message, keeping the agent on track during long operations.
- **Delegation** — Task. This spawns subagents — independent Claude instances with their own context windows that work in parallel and return summaries.

Understanding this tool set is essential because every best practice in this guide is about helping you use these tools effectively, or preventing them from being used in ways that waste your time and money.

---

2. How the Model Was Trained — and Why You Should Care

You don't need to become a machine learning researcher. But understanding the four stages of Claude's training will give you a reliable intuition for when to trust it, when to verify, and when to take the wheel yourself. Think of it like understanding how a car engine works — you don't need to rebuild one, but knowing the basics helps you drive better and diagnose problems.

Stage 1: General Pre-training — Learning What Language Looks Like

Claude started by reading an enormous amount of text — trillions of tokens from the web, books, code repositories, academic papers, and more. The training objective was simple: predict the next word. Over billions of iterations, the model internalized grammar, facts, reasoning patterns, and code syntax across dozens of programming languages.

What this gives you: A model that "knows" an extraordinary amount about how code is structured in virtually every language and framework. It has seen millions of Python functions, React components, SQL queries, and Go services.

What this doesn't give you: Correctness. The model learned what code *looks like*, not what code *does*. It has seen both correct and incorrect code, both secure and insecure patterns, and it can reproduce any of them. It learned statistical patterns, not logical proofs.

Analogy: Imagine reading every cookbook ever written. You'd know what recipes look like, what ingredients go together, and what cooking techniques exist. But you wouldn't necessarily know which recipes actually taste good — you'd just know what's been written down.

Stage 2: Code-focused Pre-training ("Mid-training") — Going Deep on Code

After general pre-training, Claude underwent additional training on curated, high-quality code corpora. This included complete repositories (not just isolated files), commit diffs showing how code evolves, pull request descriptions explaining why changes were made, and issue-to-patch pairs showing how bugs get fixed.

What this gives you: Repository-level understanding. Claude doesn't just know what a function looks like — it understands how files relate to each other through imports, how database schemas connect to API routes, how test files mirror source files. It grasps cross-file dependencies and codebase-scale patterns.

What this doesn't give you: Knowledge of *your* repository. Claude's training data was frozen at a point in time. It has never seen your codebase, your internal frameworks, or your team's conventions. It also doesn't know about APIs or frameworks that were created or changed after its training data was collected.

Stage 3: Supervised Fine-Tuning (SFT) — Learning to Be a Helpful Assistant

Next, Claude was trained on carefully written examples of ideal assistant behavior — pairs of instructions and responses crafted by humans. This taught the model to follow instructions, use tools effectively, format its responses helpfully, and behave as a useful coding assistant rather than an untrained text predictor.

What this gives you: A model that understands your intent, follows multi-step instructions, and formats output appropriately. When you say "write a test for this function," SFT is why Claude knows you want a test file, not an essay about testing.

What this doesn't give you: Reliability. SFT teaches Claude what helpful responses *look like*, but it doesn't verify that the code actually works. The model learned the *format and style* of being helpful, not the *substance* of correctness.

Stage 4: Reinforcement Learning (RLHF + RLVR) — Learning to Write Code That Works

This is the critical stage that separates modern coding AI from glorified autocomplete. In RLHF (Reinforcement Learning from Human Feedback), human evaluators compared pairs of Claude's outputs and indicated which was better. A reward model was trained from these preferences, and then Claude was optimized to produce outputs that would score highly.

But the real breakthrough for coding comes from RLVR — Reinforcement Learning with Verifiable Rewards. Instead of relying on subjective human preferences, the model generates code, and that code is *executed against test cases*. Pass or fail becomes the reward signal. This is scalable, objective, and directly tied to whether the code actually works.

What this gives you: The ability to self-correct, backtrack from wrong approaches, reason through multi-step problems, and produce code that compiles and passes tests. This is where the "thinking" capabilities emerge — the model learned that taking extra steps to verify its work leads to better rewards.

What this doesn't give you: Perfect reliability. The RL training used specific types of problems (primarily math and code with clear test cases). For problems without clear test cases — architectural decisions, security analysis, handling ambiguous requirements — the model falls back to the pattern-matching of earlier training stages.

The Practical Gap: "Looks Right" vs. "Is Right"

Here's the key insight that should shape your entire approach to working with Claude Code:

Pre-training teaches *what code looks like*. Reinforcement learning teaches *what code works*. But the bridge between these is imperfect. The model still defaults to pattern-matching when it hasn't been sufficiently trained on a specific type of problem. This creates a predictable gap:

- **Code that matches common patterns** (React components, REST APIs, CRUD operations) — high reliability, the model has seen millions of examples and been reward-trained on similar problems
- **Code that's unusual or novel** (custom algorithms, niche frameworks, complex concurrency) — lower reliability, the model is relying more on pattern-matching than verified correctness
- **Non-code decisions** (architecture, security analysis, trade-off evaluation) — the model can be helpful but you must verify with human judgment, as these weren't directly reward-trained

This is why verification is non-optional. The model doesn't have a reliable "I'm not sure" signal. It produces plausible output for everything — the question is whether that output is *correct*, and the answer depends on how close the problem is to what was heavily represented in training data.

---

3. Where the Model Is Strong vs. Weak — A Predictability Framework

Now that you understand *why* the model's capabilities vary, let's map out *where*. The model's ability at any given task is predictable from three factors:

1. **Training data density** — How much example code for this pattern exists in training data?
2. **Pattern regularity** — How template-like and repetitive is the task?
3. **RL verifiability** — Could correctness be tested during reinforcement learning training?

When all three are high, Claude is exceptional. When any drops, you need to increase your oversight proportionally.

Tasks Where Claude Excels (High Confidence, Light Oversight)

Boilerplate and CRUD — Creating data models, REST endpoints, database migrations, form components. These are the most template-like patterns in all of programming. Millions of examples in training data, extremely regular structure, easily testable. You can give Claude a data model and get a fully working API layer with high confidence.

Front-end development (React, Vue, Angular) — Component creation, state management, styling, routing. Massive training data, visual and template-based patterns, and TypeScript constrains the output further. Claude is particularly strong here because front-end code tends to follow predictable patterns.

Test generation — Writing unit tests, integration tests, and test utilities. The structure of tests is highly regular (arrange-act-assert), there are millions of examples in training data, and correctness is inherently verifiable.

Refactoring — Extracting functions, renaming symbols, modernizing syntax, converting patterns. Well-documented patterns with clear before/after examples.

Documentation and explanation — Claude is a language model at its core. Explaining code, writing docstrings, creating README files — this is its native competency.

Regex, configs, type definitions — Highly structured, deterministic outputs with constrained solution spaces.

Tasks Where Claude Is Moderate (Verify Each Step)

Bug fixing — Strong for common bug patterns (off-by-one errors, null pointer issues, incorrect conditionals). Weaker for bugs that require deep understanding of business logic or state that spans many files. The fix may be correct, but Claude might not fully understand *why* the bug exists.

Performance optimization — Claude knows common optimization patterns (memoization, lazy loading, query optimization) but struggles with problems that require profiling-driven analysis. It

might suggest the right technique but apply it to the wrong bottleneck.

API integration — Claude knows how most popular APIs work from training data, but APIs change frequently. Always fetch current documentation before implementing. Yesterday's Stripe API might not match today's.

Code review — Catches common issues reliably (unused imports, inconsistent naming, missing error handling). Misses subtle architectural concerns, implicit assumptions, and business logic errors.

Concurrency — Knows async/await patterns well at a syntactic level. Unreliable for identifying race conditions, deadlocks, or subtle ordering issues that require reasoning about multiple execution paths simultaneously.

Tasks Where Claude Struggles (Heavy Oversight Required)

Novel algorithm design — If you need a genuinely new algorithm (not a combination of known patterns), Claude has no statistical prior to draw from. It will produce something that looks like an algorithm, but the correctness is unreliable. Write the novel logic yourself; have Claude write the boilerplate, tests, and infrastructure around it.

Security-critical code — This is perhaps the most dangerous gap. Claude has seen both secure *and* insecure code in training. It can generate either with equal confidence. Authentication, encryption, access control, and payment processing must always receive expert human review and automated security scanning (SAST/DAST tools), regardless of how good Claude's output looks.

Complex distributed systems — Reasoning about failure modes across network boundaries, consensus algorithms, eventual consistency, and distributed state requires the kind of deep failure-mode analysis that has limited training examples.

Legacy systems (COBOL, Fortran, mainframes) — Minimal training data means high hallucination risk. Claude might produce syntactically plausible COBOL that does the wrong thing entirely.

Exact computation — Token prediction is not arithmetic. For any computation that needs exact results, have Claude write code that *performs* the computation rather than trying to compute it directly.

Language and Framework Strength

Strongest: Python, TypeScript/JavaScript, Java, Go, Rust, SQL — massive GitHub presence and documentation

Good: C#, Ruby (especially Rails), Swift, Kotlin, PHP (but may default to older patterns)

Unreliable: COBOL, Fortran, Ada, Assembly, APL — minimal training data, high hallucination risk

Framework rule of thumb: Mainstream frameworks dramatically outperform niche ones. React > Svelte > Qwik. Express > Hono > Fresh. Django > FastAPI > Litestar. The more popular the framework, the more training data, the better the results.

Practical Implications for Your Daily Work

Use TypeScript over JavaScript whenever possible. Type annotations constrain Claude's output space, dramatically improving generation quality. Types are essentially prompts embedded in your code — they tell Claude what shapes data can take, what functions expect and return, and what's valid. This is one of the single most impactful things you can do to improve Claude's output.

For weak areas, compensate with context. The less training data exists for your problem, the more context *you* must supply in the form of documentation, examples, and API references. If you're using a niche framework, paste its documentation or tell Claude to fetch it.

For novel problems, divide and conquer. Write the novel logic yourself. Have Claude write the boilerplate, tests, and infrastructure around it. Play to each side's strengths.

For security, never trust alone. Always run automated security tools and require expert review on authentication, encryption, access control, and payment code.

For rapidly-changing APIs, always fetch current docs. Training data is stale. Tell Claude to WebFetch the current API documentation before implementing: "Read the Stripe SDK docs at [URL] before implementing the payment flow."

Part II: Configuring Your Agent

4. CLAUDE.md — Your Highest-Leverage Investment

What It Is and Why It Matters

CLAUDE.md is loaded into **every single session** as part of the system prompt. It's the only persistent memory Claude Code has between sessions. The model is stateless — its weights are frozen, it learns nothing between conversations. Every session starts from absolute zero. CLAUDE.md is how you bridge that gap.

Think of it as onboarding documentation for a brilliant but amnesiac contractor who joins your team fresh every morning. They read your CLAUDE.md first thing, absorb the context, and start

working. Tomorrow they'll forget everything and read it again.

This makes CLAUDE.md the highest-leverage thing you can write. Every improvement you make to it improves *every future session*. Every mistake or ambiguity in it degrades *every future session*.

What to Include: The WHAT, WHY, HOW Framework

WHAT — Give Claude a map of your codebase:

```
# Project: Acme Platform
TypeScript monorepo using Turborepo. Next.js 14 frontend,
Express API, PostgreSQL with Prisma ORM.

## Structure
- /apps/web — Next.js frontend (App Router)
- /apps/api — Express REST API
- /packages/shared — Shared types and utilities
- /packages/ui — Shared React component library
- /prisma — Database schema and migrations
```

This orients Claude instantly. Without it, Claude must explore your directory structure file by file, wasting context and time.

WHY — Explain purpose and rationale:

```
## Architecture Decisions
- We use server components by default; client components only
  when interactivity requires it (why: performance, SEO)
- API routes use middleware chain pattern from /apps/api/middleware/
  (why: consistent auth, logging, error handling across all routes)
- All database queries go through Prisma, never raw SQL
  (why: type safety, migration tracking)
```

The "why" is critical because it prevents Claude from "improving" your architecture in ways that violate your design decisions. Without knowing *why* you made a choice, Claude may reasonably conclude a different approach is better and refactor toward it.

HOW — Tell Claude how to work on the project:

```
## Workflow
- Build: `turbo build`
- Test single file: `npx jest path/to/file.test.ts`
- Test all: `turbo test`
```

```
- Typecheck: `turbo typecheck`
- Lint: `turbo lint -- --fix`
- Package manager: pnpm (not npm, not yarn)
- After ANY change: run typecheck, then lint, then related tests
```

What NOT to Include — The Pruning Principle

Don't include things Claude already does correctly. If Claude never makes a specific mistake in your codebase, the rule is wasted context. Only document what Claude actually gets wrong. This is counter-intuitive — it feels like you should be thorough. But context is a scarce resource, and every unnecessary instruction reduces the model's attention on the instructions that matter.

Don't include rare or situation-specific instructions. These belong in Skills (on-demand knowledge files that load only when relevant). CLAUDE.md should contain only universally-applicable rules — things relevant to every task in every session. A rule about how to write database migrations shouldn't be in CLAUDE.md if you only write migrations once a month.

Never write negative-only rules. "Never use CommonJS" leaves Claude stuck when it needs to write a module. Always provide the alternative: "Never use CommonJS; always use ES modules with `import` / `export`." The model needs to know what *to* do, not just what to avoid.

Don't embed large reference docs. Instead, reference file paths: "For MUI component patterns, see `src/components/README.md`." Claude will read the file only when it's actually working in that area, saving context in every other session.

Never use `@-file` directives to embed full documents. These embed the entire file content on every run. Instead, point Claude to the path and let it read on demand. The difference is between always carrying a reference book versus knowing where it's shelved.

Don't duplicate what a linter does. As the HumanLayer team puts it: "Never send an LLM to do a linter's job." Code style rules like "use 2-space indentation" or "always use trailing commas" are better enforced by deterministic tools (Prettier, ESLint) via hooks. They're cheaper, faster, and 100% reliable. Using context budget on style rules that a formatter handles automatically is pure waste.

The 150-Instruction Ceiling

Research suggests frontier thinking models can reliably follow approximately 150–200 instructions with reasonable consistency. Claude Code's own system prompt already consumes about 50 of those "slots." As your CLAUDE.md grows, instruction-following quality doesn't degrade gracefully — it degrades *uniformly*. The model doesn't just ignore the newest rules; it begins ignoring all of them with increasing probability.

Target: under ~500 lines. Ruthlessly prune. Version control your CLAUDE.md. Review it like you'd review code — because it *is* code that shapes your AI's behavior. If Claude keeps ignoring a rule, your file is probably too long and the rule is getting lost.

Compensating for Training Data Gaps

If your project uses niche frameworks or languages where Claude has weaker training data (see Section 3), CLAUDE.md becomes even more critical. Include:

- Links to documentation Claude should fetch before implementing
- Code examples of your preferred patterns (few-shot examples create an overwhelming signal that overrides weaker training priors)
- Explicit style instructions for languages where Claude defaults to outdated patterns (e.g., "Always use modern PHP 8.2+ features: enums, readonly properties, fibers")

Hierarchical CLAUDE.md Files

Put general project rules in the root. Add specific rules in subdirectories:

```
/CLAUDE.md                    ← Global rules for entire project
/apps/web/CLAUDE.md           ← Frontend-specific rules
/apps/api/CLAUDE.md           ← Backend-specific rules
/packages/ui/CLAUDE.md        ← Component library conventions
```

Claude automatically merges these, prioritizing the most specific (most nested) file when working in that area. This is especially powerful in monorepos because it mirrors how senior engineers think: global principles with local constraints.

Iterate Continuously

After sessions where Claude makes mistakes, add rules to prevent those specific mistakes. After sessions where Claude follows a rule unnecessarily, remove it. Your CLAUDE.md should evolve like a living document — each iteration sharpening it based on real experience.

You can even ask Claude Code to help: "Write a hook that runs eslint after every file edit" or "Review my CLAUDE.md and suggest which rules are redundant."

---

5. The Full Extension System — Skills, Commands, Hooks, and MCP

Claude Code has four extension mechanisms, each serving a different purpose. Understanding when to use which is key to an efficient setup.

CLAUDE.md → Always-on Memory

Loaded every session. Contains universal rules. Keep it lean.

Skills → On-Demand Knowledge

Skills are markdown files in `.claude/skills/` that Claude loads only when relevant to the current task. Unlike CLAUDE.md (loaded every session), skills load on demand, saving context when they're not needed.

Use skills for specialized knowledge that doesn't apply to every session: how to create a database migration, how to write API documentation in your style, how to deploy to staging. Set `disable-model-invocation: true` for skills you only invoke manually, keeping even their descriptions out of context until needed.

When writing skills, be concise — once loaded, every token competes with conversation history. Prefer pointers to full content: reference file paths rather than embedding code snippets that will become stale.

Custom Slash Commands → Manual Workflows

Create reusable workflows in `.claude/commands/`:

```
<!-- .claude/commands/test.md -->
Write comprehensive tests for $ARGUMENTS following our testing
patterns in src/__tests__/examples/. Use Jest + RTL. Cover happy
path, error cases, and edge cases. Run tests after writing.
```

Then `/test MyComponent` executes the same reliable workflow every time. But a word of caution from practitioners: don't overdo slash commands. If you have a long list of complex custom commands that engineers must memorize, you've created an anti-pattern. The point of an agent is natural language input. Use commands as simple personal shortcuts, not as a replacement for a good CLAUDE.md.

Hooks → Deterministic Guarantees

This is the most important distinction to understand: **CLAUDE.md rules are advisory. Hooks are deterministic.** The model can forget, ignore, or deprioritize CLAUDE.md instructions — especially under context pressure. Hooks always execute, period.

Hooks fire at specific points in Claude's workflow:

- **PreToolUse** — Runs before a tool executes. Use for security gates, validation, blocking dangerous operations. Can deny the operation or modify inputs.

- **PostToolUse** — Runs after a tool completes. Use for auto-formatting, linting, logging, notifications.
- **UserPromptSubmit** — Runs when you submit a prompt, before Claude processes it. Use for prompt augmentation or validation.
- **Stop / SubagentStop** — Runs when the agent finishes. Use for cleanup or notifications.
- **SessionStart / SessionEnd** — Runs at session boundaries. Use for setup/teardown.

Example — auto-format after every TypeScript edit:

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Write|Edit",
      "hooks": [{
        "type": "command",
        "command": "npx prettier --write \"$CLAUDE_FILE_PATH\""
      }]
    }]
  }
}
```

The decision framework for choosing the right extension:

- Must it *always* happen, regardless of Claude's judgment? → **Hook**
- Should expertise apply automatically based on context? → **Skill**
- Do you want explicit control over when it runs? → **Slash Command**
- Does it apply to every single session? → **CLAUDE.md**
- None of the above? → Just prompt directly

MCP Servers → External Integrations

The Model Context Protocol connects Claude to external services: GitHub, Jira, Slack, Notion, Figma, databases, and hundreds more.

```
claude mcp add github-server
```

This enables powerful workflows like "Read the open GitHub issues and fix the highest priority one" or "Check Slack for the latest deployment status." But be aware of the cost: every MCP server adds tool definitions to every request. Three or four servers can consume significant context before you start working. Run `/mcp` to check per-server costs and disconnect servers you don't need for the current task.

Part III: Working Effectively

6. Plan Before You Build — Every Time

Why Planning Matters (The Error Compounding Problem)

Each step in an agentic chain introduces error probability. If each individual step is 95% accurate (which is optimistic for complex tasks), then after 10 steps you have only a 60% chance of an entirely correct result. After 20 steps, you're down to 36%.

Planning reduces this risk in two ways: it reduces the *total number of steps* by getting direction right upfront, and it reduces the *per-step error rate* by giving each step clearer context. Without planning, Claude will start coding immediately and often build the wrong thing confidently. With planning, Claude asks the right questions, considers edge cases, and produces code that's right the first time far more often.

This is even more critical for tasks in Claude's "moderate" or "weak" zones — where each step has a higher base error rate, the compounding gets worse faster.

The Interview Pattern (For Larger Features)

```
I want to build [brief description]. Interview me in detail using
the AskUserQuestion tool. Ask about technical implementation, UI/UX,
edge cases, concerns, and tradeoffs. Don't ask obvious questions —
dig into the hard parts I might not have considered. Keep interviewing
until we've covered everything, then write a complete spec to SPEC.md.
```

Then start a **fresh session** (`/clear`) to implement the spec. The new session has clean context focused entirely on implementation, plus a written spec it can read.

This pattern works because it forces requirements to be explicit *before* Claude commits to an implementation direction. It surfaces assumptions you didn't know you were making. Practitioners report that Claude asks questions like "Do you want social login, or just email/password?" and "Should user preferences be stored locally or in the database?" — questions that reveal hidden decisions.

The Spec-Then-Execute Pattern

1. **Session 1:** Research, plan, write to `PLAN.md` or `SPEC.md`
2. **You:** Review and refine the plan
3. **Session 2 (fresh):** "Implement the plan in PLAN.md step by step"

Each session starts with full context budget, clean history, and a clear objective. The plan is written to a file, so it survives the session boundary perfectly.

Plan Mode (Shift+Tab Twice)

This puts Claude in architect mode — it can observe, read files, and analyze, but it **cannot modify files**. In Plan Mode, Claude uses the AskUserQuestion tool to gather requirements and clarify your goals before proposing a plan. Use it to get Claude's analysis before you let it act.

This is valuable for multi-step implementations, code exploration of unfamiliar areas, and interactive development where you want to iterate on the direction before committing.

TodoWrite for Structured Execution

For complex tasks, explicitly instruct Claude to plan:

```
Before writing any code, create a detailed TODO plan with at least
10 items covering all changes needed. Mark each with priority.
Complete items one at a time, running tests after each change.
Mark items complete as you go.
```

The TODO state gets injected as a system message after each tool use, preventing goal drift during long execution loops. This is one of Claude Code's most powerful built-in mechanisms for keeping the agent on track.

Match Planning Depth to Task Difficulty

Not every task needs the same level of planning:

- **Claude's strong zone** (boilerplate, CRUD, standard patterns): Light planning, let Claude run
- **Claude's moderate zone** (bug fixing, optimization, integration): Plan first, verify each step
- **Claude's weak zone** (novel algorithms, concurrency, security): Plan extensively, write specs, human-review each change

The Progressive Disclosure Approach

For very large features, don't try to build everything in one session:

1. Phase 1: Data model and API layer
2. Phase 2: Core business logic
3. Phase 3: UI components
4. Phase 4: Integration tests and polish

Each phase gets its own session with clean context. Write progress to external files between phases so the next session picks up where you left off.

---

7. Context Management — The Core Skill

Why This Is the #1 Skill

The context window is Claude's entire working memory — everything it knows for this session. It holds system prompt, CLAUDE.md, MCP tool definitions, conversation history, file contents, command outputs, and thinking tokens. When it fills up (around 70% capacity), auto-compaction triggers: older turns get summarized, tool outputs get discarded, and information is lost.

But the problem isn't just compaction. Every token in the context window is processed on every API call. Bloated context doesn't just risk information loss — it actively degrades quality (the model has more to attend to), increases latency (more tokens to process), and costs more money (you pay per token). Context management isn't just a cost optimization — it's a quality optimization.

The Compaction Trap

When auto-compaction fires, Claude's conversation history gets summarized. If compaction happens multiple times, you get a "summary of a summary" — each round losing nuance, specific decisions, and important context. By the third or fourth compaction, Claude is working from a lossy abstraction of what actually happened, and quality degrades noticeably.

This is why the following practices matter so much:

Start Fresh Sessions Aggressively

Use `/clear` between unrelated tasks. Leftover context from a previous task is noise that degrades quality on the current task. Use `/rename` before clearing so you can `/resume` the session later if needed. Sessions are cheap — context pollution is expensive.

Practical rule: If you're about to ask Claude something unrelated to what you've been working on, clear first. The cost of re-establishing minimal context in a fresh session is far less than the quality loss from irrelevant history.

Use `/compact` Manually at Logical Breakpoints

Don't wait for auto-compact. When you finish a logical subtask, run `/compact` yourself. You can direct the compaction: `/compact Focus on the API changes and test results` to tell

Claude what information to preserve. This gives you much more control than automatic compaction.

Scope Investigations Narrowly

"Investigate the authentication system" will cause Claude to read dozens of files, filling context with verbose file contents and producing a vague summary.

Instead: "Read `src/auth/login.ts` and `src/auth/middleware.ts` and explain how the JWT token validation flow works."

If you need broad exploration, use subagents (Section 10) — they get their own context window and return only a summary to your main session.

Put Persistent Rules in CLAUDE.md, Not Conversation

Instructions given early in conversation may be lost during compaction. CLAUDE.md is reloaded fresh and survives any compaction. If something must be remembered across the entire session, it belongs in CLAUDE.md, not in a chat message.

The Write-to-File Pattern for Long Tasks

For multi-phase work, write progress to external files:

- `PROGRESS.md` — what's done, what's pending
- `DECISIONS.md` — architectural decisions made and their rationale
- `TODO.md` — remaining work items

A fresh session can read these files and resume with full signal quality. This avoids the "summary of a summary" degradation entirely, because the files contain the actual information, not a lossy compression of it.

Monitor Your Context Budget

- `/context` — See what's consuming space in the context window
- `/cost` — Track token usage and spending
- `/mcp` — Check per-MCP-server context costs
- Configure your status line to display context usage continuously

---

8. Write Prompts Like Engineering Requirements

Why Precision Matters

The model is a probability engine that follows the strongest signal. Ambiguous prompts produce ambiguous results because the model distributes probability across many plausible interpretations. Specific prompts concentrate probability on the interpretation you want.

Every correction and clarification costs tokens, context space, and risks the correction spiral (Section 11). Getting it right the first time is dramatically cheaper than iterating.

Be Specific About Inputs, Outputs, and Constraints

```
❌ "Add tests for the auth module"

✅ "Write unit tests for src/auth/login.ts using Jest and React
   Testing Library. Cover four cases: successful login, invalid
   credentials, expired token, and network timeout. Follow the
   patterns in src/auth/signup.test.ts. Mock the API calls using
   the existing src/test/mocks/api.ts helpers."
```

The second prompt constrains Claude's action space dramatically: which file, which framework, which cases, which patterns, which mocks. Every constraint reduces the number of plausible interpretations.

Reference Files Directly

Use tab-completion in your terminal to add exact file paths. Tell Claude which files to read for context and which to modify. Don't make it guess — guessing wastes context on exploration and may lead it to the wrong files.

Provide Context Claude's Training Data May Lack

- **URLs:** Paste links to docs, GitHub issues, API references — Claude will fetch and read them. Especially important for niche or new APIs.
- **Images:** Drag and drop screenshots of UI designs, error messages, or architecture diagrams
- **Piped data:** `cat error.log | claude "explain this error and fix it"`
- **Delegation:** "Use `foo-cli-tool --help` to learn about foo tool, then use it to solve X"
- **Explicit doc fetching:** "Read the Hono.js docs at [URL] before implementing the route handler"

Set Explicit Constraints

```
Output: TypeScript with strict types
Style: follow existing patterns in src/utils/
Testing: Jest, 80%+ coverage target
```

```
Dependencies: don't add new npm packages without asking
Scope: minimum change necessary — do not refactor surrounding code
```

That last constraint — scope — is one of the most important. Without it, Claude frequently "improves" surrounding code while making your requested change, creating larger diffs, more review work, and more risk.

Specify Non-Obvious Things

These seem obvious to you because you've been in the codebase. Claude is seeing it for the first time every session:

- Which testing framework and assertion library
- Functional components vs class components
- Error handling patterns (Result types? Exceptions? Error boundaries?)
- Import conventions (barrel files? Direct imports? Path aliases?)

The Devil's Advocate Pattern Before Shipping

```
I'm about to deploy this change to production. Play devil's advocate:
- What edge cases did I miss?
- What could fail under load?
- What assumptions might be wrong?
- What happens if external services are down?
- How does this handle bad data?
Be pessimistic. Think like a hacker, not a builder.
```

Thinking Budget Keywords

Claude Code maps specific words to thinking budget levels:

- `"think"` → ~4,000 thinking tokens
- `"think hard"` → more tokens
- `"think harder"` → even more
- `"ultrathink"` → ~32,000 thinking tokens (maximum budget)

Use higher thinking budgets for complex algorithmic problems, multi-file architectural decisions, debugging subtle bugs, and security review. Use lower budgets (or none) for simple tasks, to avoid wasting expensive output tokens on unnecessary deliberation.

---

9. Verification Is Non-Optional

Why This Is Architecturally Critical

The model was trained to always produce helpful, plausible output. RLHF specifically rewarded helpfulness — there's no internal "I'm not sure" signal. Even after RLVR training, the gap between "looks right" and "is right" remains significant, especially for tasks outside the model's strong zone.

Claude will generate code that looks correct, uses real-sounding API methods, follows reasonable patterns — but the methods might not exist, the logic might be subtly wrong, or the edge cases might be mishandled. This isn't a bug to be fixed; it's fundamental to how the technology works. Your job is to build verification into every workflow so the loop catches errors automatically.

Build Verification Into Every Interaction

Add to CLAUDE.md:

```
## After ANY code change:
1. Run `npm run typecheck`
2. Run `npm run lint -- --fix`
3. Run `npm run test -- --related`
If any fail, fix them before moving on. Never declare a task
complete without all three passing. Show me the passing output.
```

This creates a closed feedback loop: Claude writes code, runs verification, sees failures, fixes them, and runs verification again. The agentic loop naturally handles this iteration — you just need to tell Claude to do it.

Use Hooks for Non-Negotiable Checks

CLAUDE.md instructions are advisory — the model can ignore them, especially under context pressure. Hooks are **deterministic** and always execute:

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Write(*.ts)",
      "hooks": [{
        "type": "command",
        "command": "npx prettier --write \"$CLAUDE_FILE_PATH\""
      }]
    }, {
      "matcher": "Edit(*.py)",
      "hooks": [{
```

```
        "type": "command",
        "command": "python -m black \"$CLAUDE_FILE_PATH\""
      }]
    }]
  }
}
```

You can also use PreToolUse hooks as security gates — blocking dangerous operations before they execute, regardless of what the model decided to do.

Increase Verification for Weak Areas

For tasks in Claude's moderate/weak zones, add extra verification layers:

- **Concurrency code:** Write explicit tests for race conditions; don't trust that the happy path test is sufficient
- **Security code:** Run SAST tools as a PostToolUse hook; mandate human security review before merge
- **Niche language code:** Run the compiler/interpreter as part of the verification loop — don't rely on Claude's confidence
- **API integrations:** Test against actual (sandboxed) API endpoints, not just mocks

Git Is Your Safety Net

Claude Code creates checkpoints you can roll back to. Always:

- Work on feature branches, not main
- Commit before asking Claude to make large changes
- Use `git diff` to review what Claude actually changed before committing
- If Claude goes off the rails after several iterations, revert to a checkpoint and start fresh

Review What Claude Produces

Don't blindly trust passing tests. Read the code. Check that:

- The logic handles edge cases, not just the happy path
- Error handling is appropriate (not just `catch (e) {}`)
- The implementation matches the spec, not just the test
- No unnecessary dependencies were added
- No security issues were introduced
- The change scope matches what you asked for (no unsolicited refactoring)

10. Master Subagents for Context and Parallelism

Why Subagents Exist

Subagents solve the two biggest constraints of the agentic loop: context window limits and sequential execution. Each subagent gets its own fresh context window (up to 200K tokens) and works independently, returning only a summary to the parent. This means:

- Exploration doesn't pollute your main context
- Multiple tasks can run in parallel
- Each agent can go deep on its specific problem without crowding out others

Think of it as the difference between doing all your research and all your coding in one meeting (everything competes for attention) versus having research done by a separate team that sends you a brief.

Built-in Subagent Types

Explore — Model: Haiku (fast, cheap). Mode: read-only. Purpose: codebase search and analysis. Claude delegates to this automatically when searching. It can't modify anything, making it safe for broad exploration.

Plan — Model: Sonnet (capable). Mode: read-only. Purpose: research during Plan Mode. Gathers context before presenting you with a plan.

Task (General-purpose) — Model: Sonnet (capable). Mode: full access (read/write). Purpose: complex, multi-step operations including file editing. Used for substantial work that should be isolated from the main context.

When to Use Subagents

- The task produces verbose output you don't need in main context
- Multiple independent tasks can run in parallel
- You need deep exploration that would fill your main context
- The work is self-contained and can return a useful summary

The simplest way to trigger parallel subagents: add "use subagents" or "use 5 subagents to explore the codebase" to your request. Claude will automatically split the work.

When NOT to Use Subagents

- Simple tasks — a direct Grep is faster than spawning an agent
- Tasks that need context from the main conversation — subagents can't ask clarifying questions

- Sequential work where each step depends on the previous — parallelism doesn't help
- When you need real-time visibility — subagents work in isolation

The #1 Subagent Failure: Poor Invocation

Most subagent failures are **invocation failures** — the parent agent sends vague instructions. Subagents can't ask for clarification; they just do their best with what they receive. Ensure prompts to subagents include:

- Specific file paths to examine
- Clear success criteria
- Relevant context the subagent needs to know
- Expected output format

Parallelizing Beyond Subagents

For massive parallel work across many files, external parallelization with separate processes can be more scalable:

```
# Separate Claude instances in separate git worktrees
claude -p "in /pathA change all refs from foo to bar" &
claude -p "in /pathB change all refs from foo to bar" &
wait
```

Git worktrees are ideal for this — each worktree has its own independent file state, preventing Claude instances from interfering with each other while sharing the same Git history. The creator of Claude Code runs 5 instances simultaneously in numbered terminal tabs, using system notifications to know when any instance needs input.

Cost Optimization with Subagents

Set subagents to use a cheaper model:

```
export CLAUDE_CODE_SUBAGENT_MODEL="claude-sonnet-4-5-20250929"
```

This runs your main session on Opus while subagents use Sonnet for focused tasks, potentially cutting costs significantly.

---

11. Know the Failure Modes and Their Fixes

Understanding failure modes before they happen lets you recognize and fix them quickly. Each failure mode has a root cause in the architecture and a specific fix.

The Correction Spiral

What happens: You point out an error, Claude "fixes" it but breaks something else, you correct again, it introduces a third bug. Each correction pollutes context with wrong attempts, confused reasoning, and conflicting instructions.

Root cause: The context window fills with failed approaches. Each new attempt is influenced by the failed ones, reducing the probability of finding the correct solution.

Fix: After two failed corrections, stop. Run `/clear` and write a better initial prompt that incorporates what you learned from the failure. A fresh session with a good prompt almost always beats a polluted session with accumulated confusion.

The Infinite Exploration

What happens: You ask Claude to "investigate" something open-ended. It reads 50+ files, fills the context with verbose file contents, and produces a vague summary.

Root cause: No scope constraints on the investigation. Claude interprets "investigate" as "read everything related."

Fix: Scope investigations tightly: "Read these three files and explain X." Or delegate to a subagent so the exploration uses its own context window, returning only a summary to your main session.

The Plausible Hallucination

What happens: Claude generates code that looks correct — uses real-sounding API methods, follows reasonable patterns — but the methods don't exist or the logic is subtly wrong. Especially dangerous with unfamiliar or recently-changed APIs.

Root cause: Pattern matching from training data. The model predicts what code *should* look like based on patterns it's seen, even when those patterns don't correspond to actual APIs.

Fix: Always verify with tests, typechecking, and compilation. For APIs that may have changed since training data was collected, tell Claude to fetch current documentation first: "Fetch the Stripe SDK docs at [URL] before implementing the payment flow." Don't let Claude rely on training data for API details.

Context Degradation Over Long Sessions

What happens: After multiple compactions, Claude is working from a "summary of a summary." Quality degrades, decisions are forgotten, and instructions from early in the session get lost.

Root cause: Compaction is lossy. Each round of summarization discards detail.

Fix: Put persistent rules in CLAUDE.md (survives compaction). Write progress to external files. For long tasks, prefer the "clear and resume from files" pattern over relying on compaction: save state to `PROGRESS.md`, `/clear`, start a fresh session that reads the progress file.

The Over-Eager Refactor

What happens: You ask for a small bug fix. Claude rewrites the entire module "to be cleaner" while adding "improvements" you didn't ask for.

Root cause: The model prefers "clean" code and was trained on examples that favored comprehensive solutions. It doesn't distinguish between "fix this bug" and "make this code ideal."

Fix: Be explicit about scope: "Make the minimum change necessary to fix this bug. Do not refactor surrounding code. Do not change any code not directly related to the fix."

Scope Creep and Unsolicited Actions

What happens: Claude runs git commands when you asked it to read files, installs packages you didn't ask for, or modifies files outside the scope of the task.

Root cause: The model extrapolates what "helpful" means broadly and acts on it.

Fix: Use permission rules to constrain the action space:

```
{
  "permissions": {
    "deny": ["Bash(git push *)", "Bash(rm -rf *)", "Bash(sudo *)"],
    "ask": ["Bash(git commit *)", "Bash(npm install *)"]
  }
}
```

Stale API Knowledge

What happens: Claude generates code using API methods, parameters, or patterns that were correct at training time but have since changed.

Root cause: Training data cutoff. Claude's knowledge is frozen at a point in time.

Fix: Always WebFetch current documentation for APIs you're integrating with. Make this a habit for any external API, SDK, or service.

False Completion

What happens: Claude says "Done!" but didn't actually run the tests or verify the changes work.

Root cause: The model learned that declaring completion is part of being helpful. It doesn't always self-enforce verification steps.

Fix: Add to CLAUDE.md: "Never declare a task complete without running tests and typecheck. Show me the passing output."

The Stuck-in-a-Loop Failure

What happens: Claude keeps reading the same files, re-reading after compaction, or retrying the same failing approach without making progress.

Fix: Interrupt (Escape or Ctrl+C), steer to a different approach explicitly, or `/clear` and start over with a more specific prompt that avoids the approach that got stuck.

Part IV: Model Selection, Cost, and Team Workflow

12. Model Selection Is an Architectural Decision

Understanding the Tradeoffs

Different models have fundamentally different capabilities, costs, latency profiles, and failure modes. Using them deliberately is an engineering decision, not just a preference.

Claude Opus 4.6 — The most capable model

- Best for: Complex architecture, multi-step reasoning, ambiguous problems, large refactors, planning, novel/weak-zone tasks, security-critical review
- Costs: $15/M input, $75/M output — significantly more expensive
- Watch out for: Strong tendency to over-use subagents; slower; overkill for routine tasks
- Tip: Add to CLAUDE.md: "Use subagents only for parallel, independent workstreams. For simple searches, use Grep/Glob directly."

Claude Sonnet 4.5 — The workhorse

- Best for: Most daily coding, implementation from specs, refactoring, test writing — roughly 80% cheaper than Opus
- Handles Claude's "strong zone" tasks excellently
- Your default for implementation work

Claude Haiku 4.5 — The sprinter

- Best for: Simple tasks, quick lookups, fast iteration, Explore subagents
- Cheapest and fastest
- Watch out for: Degrades significantly with many instructions; don't use for tasks requiring many CLAUDE.md rules

Model Selection by Task Type

Task	Recommended Model	Why
Architecture and planning	Opus	Complex reasoning justifies cost
Implementation from specs	Sonnet	Strong patterns, cost-effective
Boilerplate and CRUD	Sonnet or Haiku	Patterns so strong cheaper models handle them
Novel/weak-zone tasks	Opus + high thinking budget	Maximum reasoning power needed
Code review	Sonnet (Opus for security)	Good enough for common issues
Explore subagents	Haiku	Fast, cheap, read-only

Cost-Aware Model Switching

Switch models mid-session with `/model`. Use Opus for the hard planning and architectural decisions, then switch to Sonnet for implementation. This can reduce costs by 60%+ without sacrificing quality where it matters.

For subagents:

```
export CLAUDE_CODE_SUBAGENT_MODEL="claude-sonnet-4-5-20250929"
```

Extended Thinking: Powerful but Expensive

Extended thinking is enabled by default. Thinking tokens are billed as output tokens (the expensive kind). For simple tasks, this is pure waste.

Optimize thinking costs:

- Use the `effort` parameter on Opus 4.6: low for simple tasks, high for complex ones
- Lower the thinking budget for routine work: `MAX_THINKING_TOKENS=8000`
- Disable thinking entirely for trivial lookups via `/config`
- Use the keyword system: "think" for basic, "ultrathink" for maximum reasoning

---

13. Cost Management and Token Discipline

Understanding the Cost Model

Token costs scale with context size — more context means more tokens processed per message. Claude Code optimizes via prompt caching (repeated content like system prompts) and auto-compaction, but you control the biggest cost drivers.

Average Costs to Budget For

- Average: ~$6/developer/day
- 90th percentile: ~$12/developer/day
- Monthly with Sonnet: ~$100–200/developer
- Monthly with heavy Opus: significantly more
- Use `/cost` to track actual usage

The Biggest Cost Levers

Clear between tasks. Stale context wastes tokens on every subsequent message. This is the single easiest cost saving.

Use the right model. Don't use a $75/M output model for a task a $15/M model handles fine. Sonnet for routine work, Opus for hard problems, Haiku for simple tasks.

Control extended thinking. Thinking tokens are billed as output tokens. Right-size thinking to the task complexity.

Keep CLAUDE.md lean. Every extra line is reprocessed with every message. Move specialized instructions to Skills that load on demand.

Preprocess with hooks. Instead of Claude reading a 10,000-line log file, use a hook to `grep ERROR` and feed only matching lines. Reduces context from tens of thousands of tokens to

hundreds.

Use code intelligence plugins. Language servers give Claude precise symbol navigation instead of text-based search, reducing file reads and context consumption.

Set automation safeguards. For CI/CD pipelines, configure `max_turns` and `timeout_minutes` to prevent runaway costs.

---

14. Working With Your Team

Version-Control Your Claude Code Configuration

File	Git Status	Purpose
`CLAUDE.md`	Committed, shared	Team rules and project context
`.claude/settings.json`	Committed, shared	Permissions, hooks
`.claude/settings.local.json`	Git-ignored	Personal preferences
`.claude/commands/`	Committed, shared	Team workflows
`.claude/agents/`	Committed, shared	Custom subagent definitions
`.claude/skills/`	Committed, shared	Domain knowledge

Establish Team Conventions

Document and agree on:

- Which CLAUDE.md rules are mandatory vs. recommended
- Required hooks (formatting, linting)
- Permission baselines (what's auto-allowed, what requires approval)
- Model selection guidelines (when to use Opus vs. Sonnet)
- Session hygiene practices (when to `/clear`, when to `/compact`)

Progressive Trust Escalation

Start new team members with conservative permissions (ask for everything). As trust builds, allow more auto-approved tools. Use `.claude/settings.json` to set the team baseline and let individuals customize via `settings.local.json`.

---

15. Security Considerations

Permission Discipline

The default "ask for everything" mode exists for good reason. `--dangerously-skip-permissions` is convenient but carries real risk — especially with subagents, which inherit the permission mode.

Define explicit allow/deny rules rather than blanket skip:

```
{
  "permissions": {
    "allow": ["Read", "Edit", "Bash(npm run *)", "Bash(git status)",
"Bash(git diff *)"],
    "deny": ["Bash(rm -rf *)", "Bash(sudo *)", "Bash(curl *)"],
    "ask": ["Bash(git commit *)", "Bash(git push *)", "Write"]
  }
}
```

If you choose to skip permissions for speed, mitigate the risk by running in a Docker container or isolated environment, limiting the blast radius.

Protect Sensitive Files

```
{ "deny": ["Read(.env*)", "Read(./secrets/**)",
"Write(production.config.*)"] }
```

Security Review Is Non-Negotiable for High-Risk Code

The model has seen both secure and insecure code in training and **cannot reliably distinguish** which is which. For authentication, encryption, access control, and payment processing:

- Always run SAST tools (can be automated as a PostToolUse hook)
- Mandate human security review before merge
- Use the "devil's advocate" prompt specifically for security analysis
- Never ship security-critical code without expert review, regardless of how confident the model sounds

Subagent Security

Subagents do not automatically inherit parent agent permissions. When using `bypassPermissions`, all subagents inherit this mode. Be especially careful with custom subagents that have broad tool access — they may have different system prompts and less constrained behavior than you expect.

Part V: The Bigger Picture

16. The Meta-Skill: Think Like the Architecture

Everything in this guide follows from understanding how the system works. Here's the complete mental model:

Architecture Fact	Why It Matters	What To Do
Model is stateless	Knows nothing between sessions	Invest in CLAUDE.md
Context window is finite	Degrades when full	Manage context aggressively
Training data density varies	Better at Python than COBOL	Compensate weak areas with context
Agentic loop compounds errors	Each step risks failure	Plan first, minimize steps
RL trained for test passage	Optimized for verifiable code	Always run tests
LLMs are confident when wrong	Plausible hallucinations	Verify with tests, types, lint
Compaction loses information	Summaries degrade	Use fresh sessions, write to files
Probability follows strongest signal	Ambiguity → ambiguous output	Be specific, use types
Thinking tokens cost money	Extended thinking is expensive	Right-size thinking to task
Pattern strength varies by task	CRUD ≠ novel algorithms	Match oversight to task difficulty
Tools execute locally	Real actions on your machine	Set permission boundaries
Subagents have isolated context	Exploration doesn't pollute	Use subagents for broad searches

Your New Role

The fundamental shift is this: you're no longer a developer who writes code line by line. You're an **engineering lead who directs, constrains, and verifies an autonomous agent**. Your value is in knowing:

- **What** to build — requirements, product thinking, domain knowledge
- **How** it should be architected — design decisions the model can't reliably make alone
- **What quality bar** to hold — knowing which outputs to trust and which to scrutinize
- **When to trust vs. verify** — calibrating oversight to task difficulty and model strength

The model excels at pattern-rich, well-documented, verifiable tasks. You excel at novel thinking, architectural judgment, domain knowledge, and quality assurance. The best results come from the combination — playing to each side's strengths.

Your job is to be the bridge between statistical generation and correct software. The code is generated. The engineering is still yours.

Quick Reference: Daily Checklist

Before starting work:

- [ ] Is your CLAUDE.md current? Does it reflect lessons from recent sessions?
- [ ] Are hooks configured for formatting, linting, and critical guardrails?
- [ ] Are you on the right model for today's tasks?

For each task:

- [ ] Clear context from previous tasks (`/clear`)
- [ ] Plan before building (Plan Mode or spec-to-file for complex tasks)
- [ ] Be specific: files, constraints, scope, verification steps
- [ ] Match oversight to task difficulty (light for CRUD, heavy for security)

During execution:

- [ ] Monitor context (`/context`) and cost (`/cost`)
- [ ] Compact at logical breakpoints (`/compact`)
- [ ] Use subagents for broad exploration
- [ ] Write progress to files for long tasks

Before committing:

- [ ] Tests pass, types check, linter is clean
- [ ] Read the diff — does the scope match what you asked for?
- [ ] No unsolicited refactoring or dependency additions
- [ ] Security-sensitive code gets human review

After the session:

- [ ] Did Claude make a mistake you can prevent? → Update CLAUDE.md
- [ ] Did Claude follow a rule unnecessarily? → Remove it from CLAUDE.md